

Name - Lalit D Patil

Roll No - 106

Div - B

Class - Sy

Subject - Operating

System

Sr. NO.

Title.

Remark

1.

Introduction OF O.S.

2.

System Structure.

} K/Parate
12/12/24

3.

Process Management.

4.

CPU Scheduling

5.

Process Synchronization

} K/Parate
7/13/24

Operating System

Assignment 1

1. Introduction to Operating System.

- 1) Define Operating System & List types of Operating System with their Characteristics and Functions.

→ An operating system may be viewed as an organized collection of software extensions of hardware, consisting of control routines for operating a computer and for providing an environment for execution of programs.

List Types of O.S.

- 1) Batch Operating System.
- 2) Multiprogramming
- 3) Multiprocessing System.
- 4) Multi-tasking or Time Sharing System.
- 5) Distributed System.
- 6) Networks O.S
- 7) Real Time System
- 8) Mobile O.S
- 9) Embedded O.S

Characteristics -

- 1) Multi-tasking - Multiple programs can run simultaneously

2) Multi-processing. It can take advantage of multiple processors.

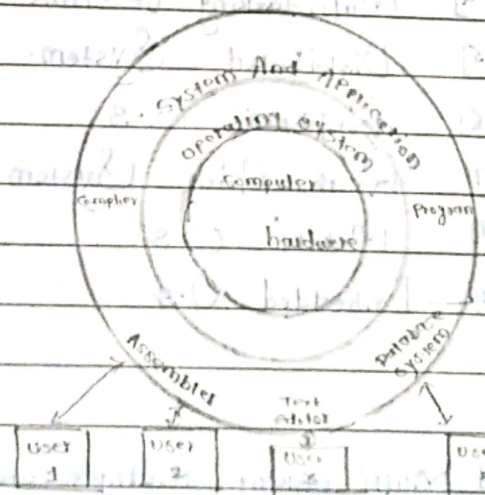
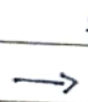
3) Multi-user. Multiple users can run programs on it simultaneously.

4) Security. It can reliably prevent application programs from directly accessing hardware devices.

Functions:

- 1) A Task Scheduler.
- 2) A Memory Manager.
- 3) A Disk Manager.
- 4) A Network Manager.
- 5) Other I/O Service Manager.
- 6) Security Manager.
- 7) Deadlocks.

Diagram of Structure of O.S



Structure of Operating System.

Q1 What is Virtual machine ? Explain its benefits.

→ Virtual machine is Virtual environment Which Simulates physical Machine. It has its Own CPU, Memory, Network Interface and Storage but they are Independent From physical hardware.

Benefits :

- 1) It provides good Security.
- 2) Virtual machine Supports the research and development of O.S.
- 3) It Solves System Compatibility problems.
- 4) It Solves the problem of System development time.

Q4 What is Spooling and Micro-kernel ?

→ Spooling - Spooling Stand for "Simultaneous peripheral Operations Online". So, In a Spooling, more than One I/O operations Can be performed Simultaneously.

Micro - Kernel - Software or Code Which Contains the required Minimum amount of Functions, data, and features to Implement an Operating System. It provides a Minimal Number of Mechanisms, which is good enough to run the most basic functions of an O.S.

Seen
12/12/23

2. System Structure

1) Which are major types of System Call? explain in detail.

→

1) Process or Job Control - process is nothing but program in execution. The execution process must be in Sequential Manner.

A running program needs to be able to halt its execution either normally or abnormally.

A process or job executing One program may want to load and execute another program.

2) Device Management - files can be thought of as abstract or Virtual devices. Thus, many of the System Calls for files are also needed for devices. If there are Multiple Users of the System however, the Users must first request the device to ensure that they have an exclusive use of it.

3) File Management - Operating System provides Several Common System Calls dealing with files. The Users may need the same sets of Operations for directories if there is a directory structure in the file system.

4) Information Maintenance - System Calls exist simply for the purpose of transferring information between the user program and the Operating System. Operating System keeps information about all of its jobs and processes and there are System Calls to access this information.

2. What is interface? State its types?

→ Operating System have User Interface (UI). This interface Can take Several Forms one is Command-line User Interface, Batch Interface, Menu Driven User Interface and Graphical User Interface (GUI).

i) Command-line User Interface - (CLI) - The Command-line User interface require a User to type Commands or press Special Keys On the keyboard to enter data and Instructions that instruct the Operating System What to do. It has to be typed One line at a time.

ii) Menu Driven User Interface - Avoids Memorizing the Commands Also it is Much easier than Commands line Interface. Here User is displayed with menus and Submenus by Clicking the appropriate option Specific tasks Carried Out.

iii) Graphical User Interface - It is a Software interface that User Interact with a pointing device, such as Mouse These are Very User friendly because they are easy to Interact with System and to execute Instruction.

iv) Form-based interface - Uses text-boxes, text-area, check-boxes etc. Create electronic form. User Completes in order to enter data into the System.

3. What is device Management and File management?

→ Device Management - files Can be thought of as abstract or Virtual devices. Thus, Many Of the System Calls for files are also needed for devices.

File Management - Operating System provides several common system calls dealing with files.

The users may need the same sets of operations for directories if there is a directory structure in the file system.

4] Explain Controls of GUI ?

→ GUI Stand For Graphical User Interface. It is a software interface that user interact with a pointing devices, such as mouse. These are very user friendly because they are easy to interact with system and to execute instructions.

Basic Components of a GUI

- 1] Pointer.
- 2] Pointing Device.
- 3] Icons.
- 4] Desktop
- 5] Windows
- 6] Menus.

5] What is process.?

→

Process is nothing but program in execution. The execution process must be in sequential manner. A running program needs to be able to halt its execution either normally (end) or abnormally (abort). A process or job executing one program may want to load and execute another program.

Q1 Explain Various types of System program.

System programs provide basic functioning to users so that they do not need to write their own environment for program development (editors, compilers) and program execution (shells).

1) File Manipulation - These programs create, delete, copy, rename, print, dump and generally manipulate files and directories.

2) Status Information - Some programs simply ask the operating system for the date, time, and amount of available memory or disk space, number of users similar status information.

3) File Modification - Several text editors may be available to create and modify contents of files stored on a disk or a tape.

4) Programming Language Support - Compilers, assemblers and interpreters for common programming languages.

5) Program Loading and execution - Once a program is assembled or compiled, it must be loaded into the memory to be executed. The operating system may provide absolute loaders, linkage editors and debugging systems in order to achieve this.

6) Applications program - In addition, most operating systems come with programs which are useful to solve some particular common problem, such as compilers, text formatters, plotting packages, database systems and so on.

6 - Deadlock

1) What is deadlock?

→

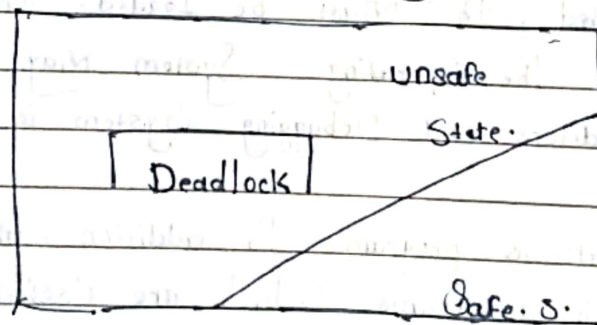
A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource required by some other process.

2) Describe Safe State in detail.

→

A system can allocate the resources to the processes in such a way so that they still avoid deadlocks then the state is called Safe State.

A sequence of process $P_1, P_2, \dots, P_n$ is a safe sequence for the current allocation state if for each P_i the resources request that P_i can still make can be satisfied by currently available resources plus the resources held by all P_j with $j < i$.



If a safe sequence does not exist, then the system is in an unsafe state, which may lead to deadlock.

3) Describe banker's algorithm With suitable example.

Banker's Algorithm is a deadlock avoidance algorithm. It is also used for deadlock detection. This algorithm tells that if any system can go into a deadlock or not by analyzing the currently allocated resources and the resources required by it in the future.

Banker's Algorithm consists in two algorithms.

1) Resource - Request Algorithm. A process makes a request of the resources. Then this algorithm checks that if the resource can be allocated or not.

$$\text{Available} = \text{Available} - \text{Request}(i)$$

$$\text{Allocation}(i) = \text{Allocation}(i) + \text{Request}(i)$$

$$\text{Need}(i) = \text{Need}(i) - \text{Request}(i)$$

2) Safety Algorithm - The Safety algorithm is applied to check whether a state is in a safe state or not.

$$\text{Work} = \text{Available}$$

$$\text{Finish}[i] = \text{false} \text{ for } i = 0, 1, \dots, n-1$$

$$\text{Finish}[i] = \text{false}$$

$$\text{Need}[i] \leq \text{Work}$$

$$\text{Work} = \text{Work} + \text{Allocation}$$

$$\text{Finish}[i] = \text{true}$$

$$\text{If } \text{Finish}[i] = \text{true} \text{ for all } i,$$

Q.2. Explain Following terms.

1) Process termination - $\div$ Eliminate deadlocks by aborting a process we use one of the two methods. In both methods, the system retains all resources allocated to the terminated processes.

Abort all deadlocked processes - This method clearly will break the deadlock cycle, but at a great expense these processes may have computed for a long time, and the result of these partial computations must be discarded and probably recomputed later.

Abort one process at a time until the deadlock cycle is eliminated.

2) Resource Preemption $\div$

1) Selecting a Victim - Deciding which resources to preempt from which processes involves many of the same decision criteria outlined above.

2) Rollback - Ideally one would like to roll back a preempted process to safe state prior to the point at which that resources was originally allocated to the process.

3) Starvation - How do you guarantee that a process will not starve because its resources are constantly being preempted.

Q1 Explain Resource allocation graph With Suitable example.

Deadlock Can be described more precisely in terms of a directed graph Called a System Resources Allocation graph.

Resources Allocation graph, processes are represented by Circle and resources are represented by boxes. Resources boxes have Some Number of dots inside indicating available Number of that resource that is Number of Instance.

$(P_i) \rightarrow [R_j]$ Process P_i is waiting for R. R_j .

$(P_i) \leftarrow [R_j]$ Process P_i has allocated R. R_j .

Q1 Enlist Various deadlock handling methods In Short.

1) Deadlock Prevention - Deadlock prevention is a Set of Method for ensuring that at least One of the necessary Condition Cannot hold.

1) Mutual exclusion.

2) Hold and wait.

3) Active approach

4) Wait time Out

5) Circular Wait

6) No preemption.

2) Deadlock Avoidance - It requires that the Operating System be given in Advance additional info. Concerning which resources a process will request and use during its lifetime With this Additional info.

5) Explain Deadlock in detail.

→

In an O.S deadlock state happens when two or more processes are waiting for the same event to happen which never happens, then we can say that these processes are involved in the deadlock.

Deadlock Handling Techniques :-

1) Deadlock Prevention :- Deadlock prevention is set of method for ensuring that at least one of the necessary condition cannot hold.

2) Deadlock Avoidance :- Require that the O.S be given in advance additional info concerning which resources a process will request and use during its lifetime.

6) What are the necessary conditions for deadlock occurs.

→

1) Mutual Exclusion :- One resource is held in a non sharable mode; that is only one process at a time can use the resources.

Ex Monitor, printer etc.

2) Hold and Wait :- A process must be holding at least one resources and waiting to acquire additional resources that are currently being held by other processes.

3) No preemption - The process which Once Scheduled will be executed till the Completion. No other process can be Scheduled by the Scheduler mean while.

4) Circular Wait - All the process must be waiting for the resources in a cyclic manner so that the last process is waiting for the resources which is being held by the first process.

Assignment - 3

C3 - Process Management

① Define the process, Context Switch, process Control Block (PCB)



1) Process :- A process is a program in execution. As the program executes the process changes state. A process is just an executing program, including the current values of the program counter, register and variables. Process execution is an alternating sequence of CPU and I/O bursts, beginning and ending with a CPU burst.

2) Context Switch :- It is a process that involves switching of the CPU from one process to another. Execution of the process that is present in the running state is suspended by the switching kernel and another process that is present in ready state is executed by the CPU.

3) Process Control Block :- Each process is represented in the operating system by a process control block (PCB) also called as TASK CONTROL BLOCK.

The O.S groups all information that it needs about a particular process into a data structure called a PCB or process descriptor.

② Explain Process States in detail.

A process is a program in execution which includes the current activity and this state is represented by the program counter and the contents of the processor's register. There is a process stack for storage of temporary data.

1) Two State Model: This represents a simple model by observing that a process either is being executed by a processor or not. It can be in two states: Running or Not running.

2) Five State Model:

i) New: The new process is created when a specific program calls from secondary memory / hard disk to primary memory RAM.

ii) Running: The process is being executed.

iii) Waiting: The process is waiting for some event to occur such as an I/O completion.

iv) Ready: The process should be loaded into the primary memory, which is ready for execution.

v) Terminated: The process has finished execution.

8-3.

3] Process Control Block (PCB) :- Each process is represented in the O.S by a process Control Block PCB also called as Task Control Block. The O.S groups all info that it needs about a particular process into a data structure called a PCB or process Descriptor.

3.4

Pointer	Process state.
Process Number	
Program Counter	
Registers	
Memory Limits.	
List of open files.	
..	

③ Explain Long term Scheduler, Short term Scheduler, Medium term Scheduler



Schedulers are special system software which handles the process scheduling in various ways.

1] Long term Scheduler :- The long term Scheduler or job Scheduler selects the process from the pool and loads

it into the main memory.

2) Short-term Scheduler: The short term S. or CPU S. selects among the processes that are ready to execute and allocate CPU to one of them. Short term S. runs very frequently.

3) Medium-term Scheduler: Medium term S. takes care of swapped out processes. If the running process needs some I/O time for completion then it needs to change its state from running to waiting.

④ What is Scheduler, explain Scheduling Queue.

Schedulers are special system software which handles the process scheduling in various ways. Main task is to select the jobs to be submitted into the system and to decide which process to run.

Scheduling Queue:- O.S maintains all p.c.s in process S. queue. The O.S maintains a separate queue for each of the process states and p.c.s of all processes in the same execution state are placed in the same queue.

1) Job queue:- All processes when enter into the system are stored in the job queue.

2) Ready Queue:- Processes in the ready state are placed in the ready queue.

3) Device Queue: Processes waiting for a device to become available are placed in device queues. There are unique device queues available for each I/O device.

⑤ Explain msg passing Systems?

Operating System provides the means for cooperating processes to communicate with each other via message passing facility. processes communicate with each other without using any kind of shared memory. This could involve a process letting another process to know that some event has occurred or transferring of data from one process to another. This communication model is Message passing Model.

Assignment 4.

4. Cpu Scheduling

1) Define Burst time, turnaround time, Dispatch Latency.

→

i) Burst time :-

ii) Turnaround time :- The Important Criterion is how long it takes to execute that process. The interval from the time of Submission of a process to the time of Completion is the turnaround time. It is the Sum of the periods Spends Waiting to get into memory, Waiting in the ready queue, executing on the CPU and doing I/O.

iii) Dispatch Latency :- The time it takes by the dispatcher to Stop One process and Start another running is known as the 'dispatch latency'.

2) What is Cpu scheduling? State the Criteria of Cpu Scheduling.

→

CPU Scheduling is the basic of multiprogrammed O.S.

The idea of Multiprogramming is relatively simple if a process (job) is waiting for an I/O request, then the CPU switches from that job to another so the CPU is always busy in Multiprogramming. In simple computer system, the CPU sits idle until the I/O request is granted.

CPU Scheduler :- The O.S must select one of the processes in the ready queue to be executed. This is carried out by Short term Scheduler.

A ready queue may be FIFO queue, priority queue, Tree, Linked list.

O.S selects from among the processes in memory that are ready to execute and allocates the CPU to one of them.

State CPU Scheduling Criteria :-

1) **CPU Utilization :-** Our main aim is to keep the CPU as busy as possible. The utilization of CPU may range from 0 to 100%. In real time, the CPU range is from 40% to 90%.

2) **Throughput :-** If CPU is busy executing processes then work is being done. One measure of work is the number of processes completed per time unit, called throughput.

3) **Turnaround time :-** The important criterion is how long it takes to execute that process. The interval from the time of submission of a process to the

time Completion is the turnaround time.

4) Waiting time :- The CPU Scheduling algorithm does not affect the amount of time during which a process executes or does I/O. The CPU Scheduling algorithm affects only the amount of time during which a process spends waiting in the ready queue. Waiting time is the addition of the periods spends waiting in the ready queue.

5) Response time :- Response time is the time from the submission of a request until the first response is produced or we can say that it is the amount of time it takes to start responding, but not the time that it takes the output that response.

Q1) Write difference between preemptive and non-preemptive Scheduling?

→

Preemptive Scheduling :- A system where processes enter the processor scheduling system and remain there sometimes executing and sometimes waiting to execute until they have finished execution. By "finished execution" we do not mean that the process terminates rather we mean that the process becomes blocked waiting for an event waiting for a message or an I/O operation to complete.

Non-Preemptive Scheduling :- The CPU has been allocated to a process, the process keeps the CPU until it releases

the CPU either by terminating or by switching to the working state.

A process is in the running state it continues to execute until it terminates or blocks itself to wait for I/O or by requesting some OS services.

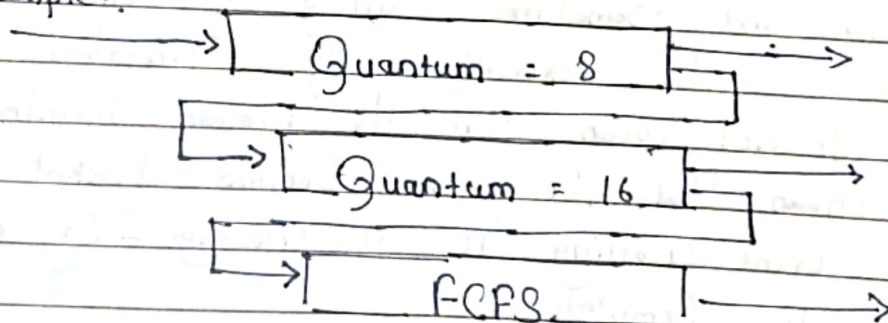
4) Explain Multilevel Feedback queue algorithm.

Multilevel Feedback queue allows a process to move between queues. The idea is to separate processes with different CPU - Burst Characteristics.

If a process uses too much CPU times, it will be moved to a lower priority queue. This leaves I/O bound and interactive processes in the higher priority queues.

If a process waits for long time in a lower priority queue, then it is moved to a higher priority queue. This form of aging prevents starvation.

Example:



5] Calculate the Average Waiting time and Average Turn-around Time by using Round Robin CPU Scheduling Algorithm. (The time Quantum is of 5 ms).

Process	CPU B.T	A.T
P ₁	28	3
P ₂	7	1
P ₃	9	2

Seen
7/5/24

Assignment - 5

5. Process Synchronization

Q.1) What is process Synchronization?

→

Process Synchronization means sharing resources by process in such a way that no two processes can share data and resources at the same time.

It is specially used in multi process system when multiple processes want to acquire same shared resource or data at the same time.

2) What is mutual exclusion?

→

If process P_1 is executing in the critical section, no other process can be executing in the critical section.

3) What is meant by deadlock and starvation of critical section problem and list its solutions.

→

The implementation of a semaphore with a waiting queue may result in a situation where more than two processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. The event in question is the execution of a signal () operation.

Starvation :- Another problem related to deadlock is indefinite blocking, or Starvation, a situation in which processes wait indefinitely within the Semaphore. Indefinite blocking may occur if we remove processes from the list associated with a Semaphore in LIFO (Last-In, First-Out) Order.

Critical Section Problem :- A race condition occurs when multiple processes or threads read and write data item so that the final result depends on the order of execution of instruction in the multiple processes.

→ A system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$. Each process has a segment of code called a Critical Section. In which the process may be changing common variables, updating a table, writing a file and so on.

4) Explain Reader, Writer Problem, Bounded buffer Problem & Dining philosopher problem in details.

→ 1) Readers & Writers Problem :- Readers and Writers is another classical problem in Con-current programming. It basically revolves around a number of processes using a shared global data structure.

A reader never modifies the shared data structure, whereas a writer may both read it and write into it. A number of readers may use the shared data structure concurrently because no matter how they are interleaved, they cannot possibly compromise its consistency.

Ex. first reader passes through MUTEX, Increments the Number of Readers and waits on Writers if Any. While reader is reading data. Semaphore MUTEX is free and WRITE is busy to allow multiple readers only.

```
/* Program readers-writers */
```

```
int readercount;
```

```
binary-semaphore mutex, write;
```

```
void reader ()
```

```
{
```

```
    while (true)
```

```
    {
```

```
        wait (mutex);
```

```
        readercount ++;
```

```
        if (readercount == 1)
```

```
            wait (write);
```

```
            signal (mutex);
```

```
            /* read data */
```

```
            wait (mutex);
```

```
            readercount --;
```

```
            if (readercount == 0)
```

```
                signal (write);
```

```
        }
```

```
    }
```

```
void Writer ()
```

```
{
```

```
    while (true)
```

```
    {
```

```
        wait (write);
```

```
        ...
```

```
        /* Write data */
```

```

    readercount = 0;
    Signal (mutex);
    Signal (write);
    initiate readers, writers
}

```

ii) Producer and Consumer With an Unbounded buffer.
Assume that buffer is of Unbounded Capacity. After the System is initialized, a producer must be the first process to run in order to provide the first item. From that pointer on, a consumer process may run whenever there is more than one item in the buffer produced but not yet consumed.

```
int n;  
binary Semaphore mutex = 1;  
general - Semaphore produced = 0;  
Void producer ()  
{
```

Scanned with OKEN Scanner


```

    }
    produce ();
    wait (mutex);
    place - in - buffer ;
    signal (mutex);
    signal (produced);
}

void consumer ()
{

```

while (true)

```

{
    wait (produced);

```

wait (mutex);

take - from - buffer ;

signal (mutex);

consume ;

}

void main

{

initiate producer , consumer ;

}

iii) Dining Philosophers problem :- The Dining philosopher problem is considered a classic synchronization problem. One simple solution is to represent each chopstick of a semaphore.

A philosopher tries to grab the chopstick by executing a wait operation on that semaphore. She releases her chopstick by executing

the Signal Operation on the appropriate Semaphore.

Ex .

do

{

Wait (chopsticks[i]);

Wait (chopsticks[(i+1) % 5]);

...

eat

S

Signal (chopsticks[i]);

Signal (chopsticks[(i+1) % 5]);

...

think

...

} while (1).

Structure philosopher.

seen
7/3/24